

DESAFIOS C#

1. Instanciar um objeto do tipo Conta e um do tipo Titular e mostrar as informações de Titular, a partir da Conta.

```
Titular t = new();
Conta c = new();
t.Nome = "Stella Leoni";
t.Cpf = "0000000000";
t.Endereco = "São Paulo";
c.Titular = t;
c.Agencia = 1;
c.NumeroDaConta = 2234;
c.Saldo = 100000;
c.Limite = 10000 ;
Console.WriteLine("Informações do Titular: ");
Console.WriteLine($"Nome: {c.Titular.Nome}");
Console.WriteLine($"CPF: {c.Titular.Cpf}");
Console.WriteLine($"Endereço: {c.Titular.Endereco}");
```

2. Desenvolver uma classe que represente um estoque de produtos, e que tenha as funcionalidades de adicionar novos produtos, e exibir todos os produtos no estoque.

```
class Produto
{
    //adicionar novos produtos
    // exibir produtos no estoque
    private double preco;
    private int estoque;
    public string Nome {get; set;}
    public string Marca {get; set;}
    public double Preco {
        get => preco;
        set
        {
            if (value > 0)
                preco = value;
            else
                preco = 10;
        }
    }

    public int Estoque {
        get => estoque;
        set
        {
            if(value > 0)
```

```

        estoque = value;
    }
    else
        estoque = 0;
    }
}

public string Descricao => $"{this.Nome} {this.Marca} - {this.reco} -
Quantidade: {this.Estoque}";
}

class EstoqueDeProdutos
{
    private List<Produto> Produto {get; set;} = new List<Produto>();
    public void AdicionarProduto(Produto EstoqueDeProduto)
    {
        Produtos.Add((produto));
        Console.WriteLine($"Produto {produto.Nome} adicionado ao estoque");
    }

    public void ExibirProdutos()
    {
        if (Produtos.Count == 0)
        {
            Console.WriteLine("Estoque vazio. Nenhum produto disponível.");
        } else
        {
            Console.WriteLine("Lista de produtos no estoque:");
            foreach (var produto in Produtos) {
                Console.WriteLine(produto.Descricao)
            }
        }
    }
}
}

```

3. Modelar o sistema de uma escola. Crie classes para Aluno, Professor e Disciplina. A classe Aluno deve ter informações como nome, idade e notas. A classe Professor deve ter informações sobre nome e disciplinas lecionadas. A classe Disciplina deve armazenar o nome da disciplina e a lista de alunos matriculados.

```

class Aluno
{
    public string Nome {get; set;}
    public int Idade {get; set;}
    public List<double> Notas {get; set; } = new List<double>();
}

```

```

class Professor
{
    public string Nome {get; set;}
    public List<string> DisciplinasLecionadas {get; set;} = new List<string>();
}

class Disciplina
{
    public string NomeDisciplina {get; set;}
    public List<Aluno> Alunos Matriculados {get; set;} = new List<Aluno>();
}

```

4. Modelar um sistema para um restaurante com classes como Restaurante, Mesa, Pedido e Cardapio. A classe Restaurante deve ter mesas que podem ser reservadas e um cardápio com itens que podem ser pedidos. Os pedidos podem estar associados a uma mesa.

```

class ProdutoRestaurante
{
    public string Nome { get; set; }
    public decimal Preco { get; set; }
}

class Mesa
{
    public int Numero { get; set; }
    public List<Pedido> Pedidos { get; set; } = new List<Pedido>();
}

class Pedido
{
    public ProdutoRestaurante Produto { get; set; }
    public int Quantidade { get; set; }
}

class Cardapio
{
    public List<ProdutoRestaurante> Itens { get; set; } = new
List<ProdutoRestaurante>();
}

class Restaurante
{
    public List<Mesa> Mesas { get; set; } = new List<Mesa>();
    public Cardapio Cardapio { get; set; } = new Cardapio();
}

```

5. Criar uma classe que representa um filme, com dados como seu título, duração e elenco. Após isso, colocá-la no namespace **Alura.Filmes**.

```
namespace Alura.Filmes;

{
class Filme
    private List<string> Elenco { get; set; }
    public string Titulo { get; set; }
    public int Duracao { get; set; }

    public Filme(string titulo, int duracao, List<string>? elenco)
    {
        if (elenco == null)
        {
            Elenco = new List<string>();
        }
        else
        {
            Elenco = elenco;
        }
        Titulo = titulo;
        Duracao = duracao;
    }

    public void AdicionarArtista(string artista)
    {
        Elenco.Add(artista);
        Console.WriteLine($"O artista {artista} foi adicionado ao elenco do filme {Titulo}");
    }
}
```

6. Criar um programa Program.cs, instanciar seus 5 filmes favoritos, guardá-los em uma lista e mostrar as suas informações no console.

```
using Alura.Filmes;

Filme filme1 = new Filme("Vingadores Ultimato", new List<string>{"Robert Downey Jr.", "Chris Evans", "Mark Ruffalo"});
Filme filme2 = new Filme("Harry Potter", new List<string>{"Daniel Radcliffe", "Emma Watson", "Rupert Grint"});
Filme filme3 = new Filme("O Diabo Veste Prada", new List<string>{"Meryl Streep", "Anne Hathaway", "Emily Blunt"});
List<Filme> filmesFamosos = new List<Filme>();
filmesFamosos.Add(filme1);
```

```

filmesFamosos.Add(filme2);
filmesFamosos.Add(filme3);

foreach(Filme f in filmesFamosos)
{
    Console.WriteLine($"Filme {f.Titulo}");
    f.ListarElenco();
    Console.WriteLine();
}

```

7. Criar uma classe Artista, que representa uma pessoa que atua em filmes, no namespace Alura.Filmes. A classe deve conter atributos como o nome, idade e uma lista de filmes onde o artista atuou.

```

namespace Alura.Filmes;
class Artista
{
    public string Nome { get; set; }
    public int Idade { get; set; }
    private List<string> Atuações { get; set; }

    public Artista(string nome, int idade)
    {
        Nome = nome;
        Idade = idade;
        Atuações = new List<Filme>();
    }

    public void AdicionarAtuação(string filme)
    {
        Atuações.Add(filme);
        Console.WriteLine($"O filme {filme.Titulo} foi adicionado às atuações do
artista {Nome}");
    }

    public void ListarAtuações()
    {
        if (this.Atuações.Count == 0)
        {
            Console.WriteLine($"O artista {this.Nome} ainda não possui atuações
registradas.");
            return;
        }

        Console.WriteLine($"Atuações do artista {this.Nome}:");
        foreach (var filme in Atuações)
        {
            Console.WriteLine($"Filme: {filme.Titulo}");
        }
    }
}

```

```
}  
  
}
```

8. Modelar um Pet Shop com classes como Pet, Dono, Consulta e médico.

```
public class Pet  
{  
    public string Nome { get; set; }  
    public int Idade { get; set; }  
    public string Especie { get; set; }  
  
    public Pet(string nome, int idade, string especie)  
    {  
        Nome = nome;  
        Idade = idade;  
        Especie = especie;  
    }  
}  
  
public class Dono  
{  
    public string Nome { get; set; }  
    public string Contato { get; set; }  
  
    public Dono(string nome, string contato)  
    {  
    }  
}  
  
public class Consulta {  
    public Pet Animal { get; set; }  
    public Dono DonoAnimal { get; set; }  
    public Medico Veterinario { get; set; }  
    public string DataConsulta { get; set; }  
  
    public Consulta(Pet animal, Dono dono, Medico veterinario)  
    {  
        Animal = animal;  
        DonoAnimal = dono;  
        Veterinario = veterinario;  
        DataConsulta = dataConsulta;  
    }  
}  
  
public class Medico  
{  
    public string Nome { get; set; }  
    public string Especialidade { get; set; }
```

```

    public Medico(string nome, string especialidade)
    {
        Nome = nome;
        Especialidade = especialidade;
    }
}

```

9. Modelar o funcionamento de uma oficina automobilística.

```

public class Veiculo
{
    public string Marca { get; set; }
    public string Modelo { get; set; }
    public int Ano { get; set; }
    public string Placa { get; set; }

    public Veiculo(string marca, string modelo, int ano, string placa)
    {
        Marca = marca;
        Modelo = modelo;
        Ano = ano;
        Placa = placa;
    }
}

public class Cliente
{
    public string Nome { get; set; }
    public string Contato { get; set; }

    public Cliente(string nome, string contato)

    public Cliente(string nome, string contato)
    {
        Nome = nome;
        Contato = contato;
    }
}

public class Mecanico
{
    public string Nome { get; set; }
    public string Especialidade { get; set; }

    public Mecanico(string nome, string especialidade)

```

```

    {
        Nome = nome;
        Especialidade = especialidade;
    }
}

public class Oficina
{
    private List<Veiculo> veiculosNaOficina;

    public Oficina()
    {
        veiculosNaOficina = new List<Veiculo>();
    }

    public void AgendarServico(Veiculo veiculo, Cliente cliente, Mecanico mecanico,
string dataAgendamento)
    {
        veiculosNaOficina.Add(veiculo);

        Console.WriteLine($"Serviço agendado para {veiculo.Placa} em
{dataAgendamento} com o mecânico {mecanico.Nome}.");
    }

    public void RealizarServico(Veiculo veiculo, Mecanico mecanico)
    {
        if (veiculosNaOficina.Contains(veiculo))
        {
            Console.WriteLine($"Serviço realizado em {veiculo.Placa} pela mecânico
{mecanico.Nome}.");
            veiculosNaOficina.Remove(veiculo);
        }
        else
        {
            Console.WriteLine($"O veículo {veiculo.Placa} não está na oficina para
realizar o serviço.");
        }
    }
}

```

10. Criar um programa Program.cs e simular o funcionamento do programa.

```

class Program
{
    static void Main()
    {
        Veiculo meuCarro = new Veiculo("Honda", "Civic", 2020, "ACR1234");
    }
}

```



```

    Cliente cliente = new Cliente("Carlos", "328493428");
    Mecanico mecanico = new Mecanico("José", "Mecânica Especializada");
    Oficina oficina = new Oficina();

    oficina.AgendarServico(meuCarro, cliente, mecanico, "2026-12-18");
    oficina.RealizarServico(meuCarro, mecanico);
}
}

```

11. Escrever um programa que funcione como uma calculadora, que pode realizar as 4 operações básicas, além de calcular raiz quadrada e potências. O usuário deve entrar com dois números e um símbolo que represente a operação a ser feita.

```

public class Calculadora
{
    public static double Calcular(double numero1, double numero2, char operacao)
    {
        double resultado = 0;
        switch (operacao)
        {
            case '+':
                resultado = Somar(numero1, numero2);
                break;
            case '-':
                resultado = Subtrair(numero1, numero2);
                break;
            case '*':
                resultado = Multiplicar(numero1, numero2);
                break;
            case '/':
                resultado = Dividir(numero1, numero2);
                break;
            case '^':
                resultado = Potencia(numero1, numero2);
                break;
            case 'r':
                resultado = RaizQuadrada(numero1);
                break;
            default:
                Console.WriteLine("Operação inválida.");
                break;
        }
        return resultado;
    }

    private static double Somar(double a, double b)
    {

```

```

        return a + b;
    }

    private static double Subtrair(double a, double b)
    {
        return a - b;
    }

    private static double Multiplicar(double a, double b)
    {
        return a * b;
    }

    private static double Dividir(double a, double b)
    {
        if (b != 0)
            return a/b;
        else
        {
            Console.WriteLine("Erro: Divisão por zero.");
            return 0;
        }
    }

    private static double Potencia(double a, double b)
    {
        return Math.Pow(a, b);
    }

    private static double RaizQuadrada(double a)
    {
        return Math.Sqrt(a);
    }
}

```

12. Criar uma hierarquia de classes representando formas geométricas, como Quadrado, Círculo e Triângulo. Utilize herança para criar uma classe base chamada FormaGeometrica, que contenha métodos para calcular a área e o perímetro de uma forma.

```

abstract class FormaGeometrica
{
    public abstract double CalcularArea();
    public abstract double CalcularPerimetro();
}

class Quadrado : FormaGeometrica
{

```

```

    public double Lado { get; set; }
    public override double CalcularArea()
    {
        return ReadLine;ado * Lado
    }

    public override double CalcularPerimetro()
    {
        return 4 * Lado;
    }
}

class Circulo : FormaGeometrica
{
    public double raio { get; set; }

    public override double CalcularArea()
    {
        return Math.PI * Raio * Raio;
    }

    public override double CalcularPerimetro()
    {
        return 2 * Math.PI * Raio;
    }
}

class Triangulo : FormaGeometrica
{
    public double Base { get; set; }
    public double Altura { get; set; }

    public override double CalcularArea()
    {
        return 0.5 * Base * Altura;
    }

    public override double CalcularPerimetro()
    {
        return Base + Altura + Math.Sqrt(Base * Base + Altura * Altura)
    }
}

```

13. Crie uma hierarquia de classes representando funcionários de uma empresa. Utilize herança para criar classes como Gerente, Programador e Analista. Cada

classe deve ter propriedades específicas, além das propriedades comuns a todos os funcionários, como Nome e Salário.

```
class Funcionario
{
    public string Nome { get; set; }
    public double Salario { get; set; }
}

class Gerente : Funcionario
{
    public string Setor { get; set; }
}

class Programador : Funcionario
{
    public string Programacao { get; set; }
}

class Analista : Funcionario
{
    public string Dados { get; set; }
}
```

14. Criar uma hierarquia de classes representando contas bancárias, como ContaCorrente e ContaPoupanca. Utilize herança e o conceito de métodos virtuais para implementar um método CalcularSaldo que retorne o saldo atual da conta.

```
class ContaBancaria
{
    protected double Saldo { get; set; }
    public virtual void Depositar(double valor)
    {
        Saldo += valor;
    }

    public void Sacar(double valor)
    {
        Saldo -= valor;
    }

    public void double CalcularSaldo()
    {
        return Saldo;
    }
}

class ContaCorrente : ContaBancaria
```

```

{
    private double TaxaManutencao { get; set; }
    public override void Sacar(double valor)
    {
        base.Sacar(valor + TaxaManutencao);
    }
}

class ContaPoupanca : ContaBancaria
{
    private double TaxaRendimento { get; set; }

    public override double CalcularSaldo()
    {
        return base.CalcularSaldo() * (1 + TaxaRendimento);
    }
}

```

15. Criar uma hierarquia de classes representando animais, como Mamifero, Ave e Peixe. Utilize herança e o conceito de métodos virtuais para implementar um método EmitirSom que represente o som característico de cada tipo de animal.

```

class Animal
{
    public virtual string EmitirSom()
    {
        return "Som genérico de animal";
    }
}

class Mamifero : Animal
{
    public override string EmitirSom()
    {
        return "Som de mamífero";
    }
}

class Ave : Animal
{
    public override string EmitirSom()
    {
        return "Som de ave";
    }
}

class Peixe : Animal

```

```
{
    public override string EmitirSom()
    {
        return "Som de peixe";
    }
}
```

16. Criar uma hierarquia de classes representando produtos eletrônicos, como Smartphone, Tablet e Laptop. Utilize herança e o conceito de métodos virtuais para implementar um método `ExibirInformacoes` que retorne informações específicas de cada produto.

```
class ProdutoEletronico
{
    public string Modelo { get; set; }
    public double Preco { get; set; }

    public virtual string ExibirInformacoes()
    {
        return $"Modelo: {Modelo}, Preço: {Preco:C}";
    }
}

class Smartphone : ProdutoEletronico
{
    public string SistemaOperacional { get; set; }

    public override string ExibirInformacoes()
    {
        return $"{base.ExibirInformacoes()}, SO: {SistemaOperacional}";
    }
}

class Tablet : ProdutoEletronico
{
    public string TipoTela { get; set; }

    public override string ExibirInformacoes()
    {
        return $"{base.ExibirInformacoes()}, Tela: {TipoTela}";
    }
}

class Laptop : ProdutoEletronico
{
    public string Processador { get; set; }

    public override string ExibirInformacoes()
```

```

    {
        return $"{base.ExibirInformacoes()}, Processador: {Processador}";
    }
}

```

17. Criar uma interface chamada IForma que declare métodos para calcular a área e o perímetro de uma forma geométrica. Implemente esta interface em duas classes: Circulo e Retangulo.

```

internal interface IForma
{
    double CalcularArea();
    double CalcularPerimetro();
}

public class Circulo : IForma
{
    public double Raio { get; set; }

    public double CalcularArea()
    {
        return Math.PI * Math.Pow(Raio, 2);
    }

    public double CalcularPerimetro()
    {
        return 2 * Math.PI * Raio;
    }
}

public class Retangulo : IForma
{
    public double Comprimento { get; set; }
    public double Largura { get; set; }

    public double CalcularArea()
    {
        return Compriment * Largura;
    }

    public double CalcularPerimetro()
    {
        return 2 * (Comprimento + Largura);
    }
}

```

18. Criar duas interfaces adicionais, IPilotavel e IVoavel. Implemente essas interfaces na classe Veiculo.

```

public interface IPilotavel
{
    void Pilotar();
}

public interface IVoavel
{
    void Voar();
}

public class Veiculo : IPilotavel, IVoavel
{
    public void Pilotar()
    {
        Console.WriteLine("Veículo está pilotando");
    }

    public void Voar()
    {
        Console.WriteLine("Veículo está voando");
    }
}

```

19. Criar uma interface chamada IPagavel com um método CalcularPagamento. Implemente essa interface em duas classes, Produto e Servico. O método CalcularPagamento deve retornar o valor total a ser pago, levando em consideração a quantidade para produtos e a taxa horária para serviços.

```

public interface IPagavel
{
    decimal CalcularPagamento();
}

public class Produto : IPagavel
{
    public string Nome { get; set; }
    public decimal PrecoUnitario { get; set; }
    public int Quantidade { get; set; }

    public decimal CalcularPagamento()
    {
        return PrecoUnitario * Quantidade;
    }
}

public class Servico : IPagavel
{
    public string Nome { get; set; }

```



```

    public decimal TaxaHoraria { get; set; }
    public int HorasTrabalhadas { get; set; }

    public decimal CalcularPagamento()
    {
        return TaxaHoraria * HorasTrabalhadas;
    }
}

```

20. Criar uma interface chamada INotificavel com um método EnviarNotificacao. Implemente essa interface em duas classes, Email e SMS. O método EnviarNotificacao deve exibir mensagens diferentes para cada tipo de notificação.

```

public interface INotificavel
{
    void EnviarNotificacao();
}

public class Email : INotificavel
{
    public string EnderecoEmail { get; set; }

    public void EnviarNotificacao()
    {
        Console.WriteLine($"Enviando e-mail para {EnderecoEmail}: Notificacao importante!");
    }
}

public class SMS : INotificavel
{
    public string NumeroTelefone { get; set; }

    public void EnviarNotificacao()
    {
        Console.WriteLine($"Enviando SMS para {NumeroTelefone}: Notificacao importante!");
    }
}

```

21. Criar uma interface chamada IArmazenavel com métodos Salvar e Recuperar. Implemente essa interface em duas classes, Arquivo e BancoDeDados. Os métodos Salvar e Recuperar devem exibir mensagens simulando a ação de salvar e recuperar dados.

```

public interface IArmazenavel
{
    void Salvar();
    void Recuperar();
}

```

```

}

public class Arquivo : IArmazenavel
{
    public string NomeArquivo { get; set; }

    public void Salvar()
    {
        Console.WriteLine($"Salvando dados no arquivo {NomeArquivo}")
    }
    public void Recuperar()
    {
        Console.WriteLine($"Recuperando dados no arquivo {NomeArquivo}")
    }
}

public class BancoDeDados : IArmazenavel
{
    public string NomeBancoDados { get; set; }

    public void Salvar()
    {
        Console.WriteLine($"Salvando dados no banco de dados {NomeBancoDados}")
    }
    public void Recuperar()
    {
        Console.WriteLine($"Recuperando dados no banco de dados {NomeBancoDados}")
    }
}

```

22. Escrever um programa que solicite dois números a e b lidos do teclado e realize a divisão de a por b. Caso essa operação não seja possível, mostrar uma mensagem no console que deixe claro o erro ocorrido.

```

try
{
    Console.Write("Digite o numerador: ");
    int numerador = int.Parse(Console.ReadLine());
    Console.Write("Digite o denominador: ");
    int denominador = int.Parse(Console.ReadLine());
    int resultado = numerador / denominador;
    Console.WriteLine($"Resultado: {resultado}");
}
catch (DivideByZeroException ex)
{
    Console.WriteLine($"Erro: na matemática não é permitida a divisão por 0.")
}

```

23. Declarar uma lista de inteiros e tente acessar um elemento em um índice inexistente. Tratar a exceção apropriada.

```
try
{
    List<int> numeros = new List<int> { 1, 2, 3 };
    Console.WriteLine($"Elemento no índice 5: {numeros[5]}");
}
catch (ArgumentOutOfRangeException ex)
{
    Console.WriteLine($"Erro: {ex.Message}")
}
```

24. Criar uma classe simples com um método e chame esse método em um objeto nulo. Tratar a exceção de método em objeto nulo.

```
try
{
    MinhaClasse objetoNulo = null;
    objetoNulo.Meumetodo();
}
catch (NullReferenceException ex)
{
    Console.WriteLine($"Erro: {ex.Message}")
}
```

25. Modelar e desserializar a classe Filme

```
//Classe Filme
namespace Filmes;

using System.Text.Json;
using System.Text.Json.Serialization;

internal class Filme
{
    [JsonPropertyName("title")]
    public string? Titulo { get; set; }

    [JsonPropertyName("year")]
    public string? Ano { get; set; }

    [JsonPropertyName("crew")]
    public string? Elenco { get; set; }

    [JsonPropertyName("imDbRating")]
    public string? Nota { get; set; }
}
```

```

        public string FichaTecnica => $"\\n\\nTitulo: {Titulo} ({Ano}) - Nota:
{Nota}\\nElenco: [{Elenco}]\\n\\n";
    }

//Program.cs
namespace Filmes;

using System.Text.Json;
using System.Text.Json.Serialization;

using (HttpClient client = new HttpClient())
{
    try
    {
        string resposta = await
client.GetStringAsync("https://raw.githubusercontent.com/ArthurOcFernandes/Exerc-ci
os-C-/curso-4-aula-2/Jsons/TopMovies.json");
        var filmes = JsonSerializer.Deserialize<List<Filme>>(resposta!);
        foreach (var filme in filmes)
        {
            Console.WriteLine(filme.FichaTecnica)
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Temos um problema: {ex.Message}");
    }
}

```

26. Modelar e desserializar a classe Carro

```

//Classe Carro
namespace Carros;

using System.Text.Json;
using System.Text.Json.Serialization;

internal class Carro
{
    [JsonPropertyName("marca")]
    public string? Marca { get; set; }

    [JsonPropertyName("modelo")]
    public string? Modelo { get; set; }

    [JsonPropertyName("ano")]

```

```

    public int? Ano { get; set; }

    [JsonPropertyName("tipo")]
    public string? Tipo { get; set; }

    public string Informacoes => $"
    \n Carro {Marca} {Modelo} {Ano} {Tipo}";
}

//Program.cs
namespace Carros;

using System.Text.Json;
using System.Text.Json.Serialization;

using (HttpClient client = new HttpClient())
{
    try
    {
        string resposta = await
client.GetStringAsync("https://github.com/ArthurOcFernandes/Exerc-cios-C-/raw/curso
-4-aula-2/Jsons/Carros.json");
        var carros = JsonSerializer.Deserialize<List<Carro>>(resposta)!;
        foreach (var carro in carros)
        {
            Console.WriteLine(carro.Informacoes)
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Temos um problema: {ex.Message}");
    }
}

```

27. Dada uma lista de números, criar uma consulta LINQ para retornar apenas os elementos únicos da lista.

```

List<int> numeros = new List<int> { 1, 2, 3, 2, 4, 5, 3, 6 , 7, 7, 8};
var numerosUnicos = numeros.Distinct()

Console.WriteLine("Números únicos na lista:");
foreach (var in numero in numerosUnicos)
{
    Console.Write(numero + " ");
}

```

28. Dada uma lista de livros com título, autor e ano de publicação, criar uma consulta LINQ para retornar uma lista com os títulos dos livros publicados após o ano 2000, ordenados alfabeticamente.

```
class Livro
{
    public string Titulo { get; set;}
    public string Autor { get; set; }
    public int AnoPublicacao { get; set; }
}

List<Livro> livros = new List<Livro>
{
    new Livro { Titulo = "Aprendendo LINQ", Autor = "João Silva", AnoPublicacao = 2005 },
    new Livro { Titulo = "Programação em C#", Autor = "Ana Oliveira", AnoPublicacao = 2010 },
    new Livro { Titulo = "Algoritmos e Estruturas de Dados", Autor = "Carlos Santos", AnoPublicacao = 1998 },
    new Livro { Titulo = "Introdução à Inteligência Artificial", Autor = "Mariana Costa", AnoPublicacao = 2021 },
    new Livro { Titulo = "Design Patterns", Autor = "Paulo Rocha", AnoPublicacao = 2002 }
};

var titulosLivros = livros.Where(l => l.AnoPublicacao > 2000).OrderBy(l => l.Titulo).Select(l => l.Titulo);
Console.WriteLine("Títulos de livros publicados após 2000, ordenados alfabeticamente:");
foreach (var titulo in titulosLivros)
{
    Console.WriteLine(titulo);
}
```

29. Dada uma lista de produtos com nome e preço, criar uma consulta LINQ para calcular o preço médio dos produtos.

```
class Produto
{
    public string Nome { get; set;}
    public decimal Preco { get; set; }
}

List<Produto> produtos = new List<Produto>
{
    new Produto { Nome = "Computador", Preco = "1400"},
    new Produto { Nome = "Celular", Preco = "3000"},
    new Produto { Nome = "Relógio", Preco = "300"}
}
```

```
};
var precoMedio = produtos.Average(p => p.Preco);
Console.WriteLine($"Preco médio dos produtos: {precoMedio}");
```

30. Dada uma lista de strings, criar uma consulta LINQ para ordenar as palavras por comprimento e retornar apenas aquelas que têm mais de 3 caracteres.

```
List<string> palavras = new List<string>{"professor", "batata", "arroz",
"caminhão", "paralelepipedo"};
var palavrasFiltradas = produtos.Where(p => p.Length > 3).OrderBy(p => p.Length);
Console.WriteLine($"Palavras com mais de 3 caracteres, ordenadas por comprimento");
foreach (var palavra in palavrasFiltradas)
{
    Console.Write(palavra + " ");
}
```

31. Criar um programa que permite ao usuário inserir informações de uma pessoa (nome, idade, e e-mail), serializa essas informações em formato JSON e salva em um arquivo.

```
class Pessoa
{
    public string Nome { get; set; }
    public int Idade { get; set; }
    public string Email { get; set; }
}

class Program
{
    static void Main()
    {
        Pessoa pessoa = new Pessoa();
        Console.WriteLine("Digite o nome: ");
        pessoa.Nome = Console.ReadLine();
        Console.WriteLine("Digite a idade: ");
        pessoa.Idade = Console.ReadLine();
        Console.WriteLine("Digite o Email: ");
        pessoa.Email = Console.ReadLine();

        string json = JsonSerializer.Serialize(pessoa);
        string nomeDoArquivo = $"pessoa.json";

        File.WriteAllText(nomeDoArquivo, json);
        Console.WriteLine($"O arquivo JSON foi criado com sucesso!");
        Console.WriteLine(Path.GetFullPath(nomeDoArquivo));
    }
}
```

32. Criar um programa que lê um arquivo JSON contendo informações de várias pessoas, desserializa essas informações em uma lista e exibe na tela.

```
class Pessoa
```

```
{
    public string Nome { get; set; }
    public int Idade { get; set; }
    public string Email { get; set; }
}

class Program
{
    static void Main()
    {
        string nomeDoArquivo = $"pessoa.json";
        if (File.Exists(nomeDoArquivo))
        {
            string json = File.ReadAllText(nomeDoArquivo);
            List<Pessoa> pessoas = JsonSerializer.Deserialize<List<<Pessoa>>>(json);

            Console.WriteLine("Informações das pessoas: ");

            foreach (Pessoa pessoa in pessoas)
            {
                Console.WriteLine($"Nome: {pessoa.Nome}, Idade: {pessoa.Idade},
E-mail: {pessoa.Email}");
            }
        }
        else
        {
            Console.WriteLine($"O arquivo {nomeDoArquivo} não existe");
        }
    }
}
```